NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 3: String Manipulation, Guess-and-Check, Approximate Solutions, and Bisection

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 3 at a glance

Lecture 7

String manipulation

- indexing, slicing, and immutability;
- methods for cleaning and inspecting text;
- simple engineering records and identifiers.

Lecture 8

Guess-and-check and approximation

- exact versus approximate answers;
- step size, tolerance, and stopping;
- progress, failure modes, and cost.

Lecture 9

The bisection method

- roots, brackets, and sign changes;
- repeated interval halving;
- reliable engineering root finding.

Represent the data clearly → Search systematically → Accept a justified approximation → Verify the result

NAZARBAYEV UNIVERSITY

LECTURE 7

String Manipulation for Engineering Data

Reading, cleaning, validating, and communicating text without losing its engineering meaning

NAZARBAYEV UNIVERSITY

Lecture 7 learning goals

By the end of this lecture, you should be able to:

1. treat a string as an ordered, immutable sequence;
2. use positive and negative indexing;
3. extract substrings using slicing;
4. clean and inspect text with common methods;
5. split a simple engineering record and convert numerical text;
6. normalize input without erasing identifiers;
7. test representative, boundary, and malformed records.

This week's first transition

Programs often interpret labels, units, IDs, and sensor messages before numerical calculation.

NAZARBAYEV UNIVERSITY

Retrieval warm-up: predict before running

```
raw_unit = "  MPa  "
unit = raw_unit.strip().lower()

print(unit)
print(unit == "mpa")

for character in "P7":
    print(character)
```

Four questions

1. What does `strip()` remove?
2. Does `lower()` change `raw_unit`?
3. What Boolean value is printed?
4. How many loop passes occur?

Individual - 3 min

NAZARBAYEV UNIVERSITY

Opening engineering problem: a sensor message arrives as text

Raw message

␣␣T-07 , 23.50 , C ␣

- Which sensor produced the reading?
- What numerical value was measured?
- Which unit was reported?
- Are the spaces part of the intended information?

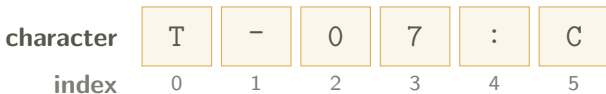
Before calculating

The program must locate separators, remove accidental whitespace, preserve the component ID, convert numerical text, and retain the unit.

Text is engineering data

NAZARBAYEV UNIVERSITY

A string is an ordered sequence of characters



Order matters

`"T-07:C"` and `"C:T-07"` contain similar characters but encode different information.

Length matters

`len("T-07:C")` is 6, so the valid positive indices are 0 through 5.

NAZARBAYEV UNIVERSITY

Indexing retrieves one character

```
label = "T-07:C"

first = label[0]
second = label[1]
unit_code = label[5]

print(first)
print(second)
print(unit_code)
```

Zero-based indexing

- index 0 is the first character;
- index 1 is the second character;
- for length n, the last positive index is $n - 1$.

Boundary failure

`label[6]` raises `IndexError`: the position does not exist.

NAZARBAYEV UNIVERSITY

Negative indices count from the end

```
label = "T-07:C"

print(label[-1])
print(label[-2])
print(label[-3])
```

Expression	Character	Meaning
`label[-1]`	C	last
`label[-2]`	:	second-to-last
`label[-3]`	7	third-to-last

Why useful?

Negative indexing is convenient when a suffix has a fixed meaning but the full label length varies.

Slicing extracts a substring

```
record = "T-07,23.50,C"

sensor_id = record[0:4]
value_text = record[5:10]
unit = record[11:12]

print(sensor_id)
print(value_text)
print(unit)
```

Half-open interval

`s[start:stop]` includes `start` but excludes `stop`.

Why exclude the stop?

The slice length is `stop - start`. Adjacent slices can meet without overlapping:

$$s[0:4] \quad s[4:8].$$

NAZARBAYEV UNIVERSITY

Useful slice shortcuts

Expression	Meaning	Example for `s = "PUMP-107A"`
`s[:4]`	from the beginning through index 3	`"PUMP"`
`s[5:]`	from index 5 to the end	`"107A"`
`s[-4:]`	last four characters	`"107A"`
`s[::2]`	every second character	`"PMP0A"`
`s[::-1]`	reverse the string	`"A701-PMUP"`

Slicing is forgiving

A slice that extends beyond the end usually stops safely. Direct indexing beyond the end raises an error.

NAZARBAYEV UNIVERSITY

Activity: predict the slices before running

```
code = "VALVE-204B"

print(code[:5])
print(code[6:9])
print(code[-1])
print(code[-4:])
print(code[1:8:2])
```

For each line

1. mark the selected indices;
2. write the exact output;
3. state whether the stop index is included.

Pairs - 4 min

Strings are immutable

This does not work

```
code = "pump-07"
code[0] = "P"
```

Python raises a `TypeError` because an existing string character cannot be replaced in place.

Create a new string

```
code = "pump-07"
corrected = "P" + code[1:]

print(code)
print(corrected)
```

Mental model

A variable can be rebound to a new string, but the original string object is not modified character by character.

NAZARBAYEV UNIVERSITY

String methods return transformed strings

```
raw = "  Pump-07  "

clean = raw.strip()
lowercase = clean.lower()
uppercase = clean.upper()

print(raw)
print(clean)
print(lowercase)
print(uppercase)
```

Method call anatomy

```
object.method(arguments)
```

- the dot selects an operation associated with the object;
- parentheses perform the call;
- the returned string should usually be stored.

NAZARBAYEV UNIVERSITY

Normalize human-entered text deliberately

```
raw_material = input("Material: ")
material = raw_material.strip().lower()

if material == "steel":
    print("Steel selected")
else:
    print("Different material")
```

Helpful normalization

Treat `" Steel "`, `"STEEL"`, and `"steel"` as the same human-entered material name.

Do not normalize blindly

`P-007` and `P-7` may refer to different assets. Removing zeros or punctuation can destroy identity.

NAZARBAYEV UNIVERSITY

Inspect prefixes, suffixes, and membership

```
file_name = "test_run_07.csv"
component = "PUMP-204B"

print(file_name.endswith(".csv"))
print(component.startswith("PUMP"))
print("204" in component)
print("TEMP" not in component)
```

Boolean questions about text

- Is this the expected file type?
- Does this code use the required equipment prefix?
- Does a particular token occur?

Case-sensitive by default

`"pump" in "PUMP-204B"` is `False`.

Replace, find, and count

```
message = "P-07|245.6|kPa"

csv_like = message.replace("|", ",")
separator_index = message.find("|")
separator_count = message.count("|")

print(csv_like)
print(separator_index)
print(separator_count)
```

Interpret carefully

- `replace()` returns new text;
- `find()` returns the first index, or -1 if absent;
- `count()` reports occurrences.

Do not confuse -1 with a valid location

Test membership or compare the result before using the index.

NAZARBAYEV UNIVERSITY

Split a structured record into fields

```
raw_record = "  T-07 , 23.50 , C  "

sensor_id, value_text, unit = raw_record.split("
    ,")

sensor_id = sensor_id.strip()
value_c = float(value_text)
unit = unit.strip().upper()

print(sensor_id)
print(value_c)
print(unit)
```

What `split()` does

It divides a string at each separator and returns the resulting pieces. Here, three pieces are assigned to three names.

Structural assumption

This code expects exactly two commas and three fields. A missing or extra field must be handled in a more robust program.

NAZARBAYEV UNIVERSITY

Join cleaned fields into a standard label

```
area = "north"
equipment = "pump"
number = "007"

parts = [area.upper(), equipment.upper(),
    number]
standard_id = "-".join(parts)

print(standard_id)
```

Output

```
NORTH-PUMP-007
```

Preview of a collection

The brackets hold several string values. Lists will be studied formally later; here they simply provide the pieces that `join()` combines.

NAZARBAYEV UNIVERSITY

A loop can inspect every character

```
component_id = "P-204B"
digit_count = 0

for character in component_id:
    if character.isdigit():
        digit_count += 1

print(f"Digits: {digit_count}")
```

Character tests

Useful methods include:

- `isdigit()`;
- `isalpha()`;
- `isalnum()`;
- `isspace()`.

Ask what is valid

A component-ID rule is an engineering data specification, not merely a Python exercise.

NAZARBAYEV UNIVERSITY

Integrated example: clean, parse, and report

```
raw = "  P-204 , 247.6 , kPa  "

component, value_text, unit = raw.split(",")
component = component.strip().upper()
pressure_kpa = float(value_text)
unit = unit.strip()

valid_prefix = component.startswith("P-")
valid_unit = unit.lower() == "kpa"

print("PRESSURE RECORD")
print(f"Component: {component}")
print(f"Pressure:  {pressure_kpa:.1f} {unit}")
print(f"Prefix valid: {valid_prefix}")
print(f"Unit valid:   {valid_unit}")
```

Workflow

1. separate fields;
2. clean each field;
3. convert the numerical field;
4. check expected structure;
5. print evidence, not only a number.

NAZARBAYEV UNIVERSITY

Debugging clinic: four common string mistakes

Mistake	Faulty idea	Repair question
Missing method call	`clean = raw.strip`	Did you include parentheses?
Ignored return value	`raw.strip()` then use `raw`	Where is the transformed string stored?
Invalid index	`code[len(code)]`	What is the last valid index?
Direct mutation	`code[0] = "P"`	How can slicing construct a new string?
Wrong type	`"23.5" + 1.0`	Which conversion is needed before arithmetic?

Debug one fact at a time

Print both the value and its type. Use `repr(text)` when invisible spaces or newline characters may be present.

Testing a text-processing rule

Representative cases

- normal record: `P-204,247.6,kPa`;
- different capitalization;
- surrounding spaces;
- multi-digit and leading-zero IDs.

Boundary and invalid cases

- empty string;
- missing separator;
- extra separator;
- nonnumeric value field;
- unexpected unit.

Engineering interpretation

A program that accepts malformed data without warning can produce a precise-looking but meaningless calculation.

NAZARBAYEV UNIVERSITY

Exit ticket: from raw text to trustworthy information

```
raw = "  t-12 | 88.4 | c  "
clean = raw.strip().replace("|", ",")

print(clean)
print(clean[:4])
print(clean.endswith("c"))
```

Submit three responses

1. Predict all outputs.
2. State one assumption about the record format.
3. Name one invalid case that should be tested.

Individual - 3 min

Key message

String manipulation should make engineering information more explicit, not silently change its meaning.

NAZARBAYEV UNIVERSITY

LECTURE 8

Guess-and-Check and Approximate Solutions

Turning repeated trials into a controlled numerical search with explicit accuracy and stopping rules

NAZARBAYEV UNIVERSITY

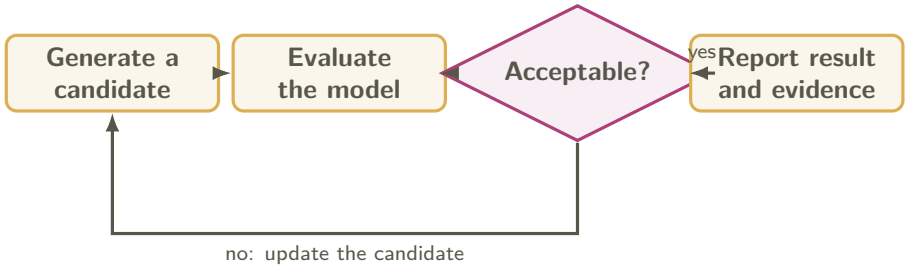
Lecture 8 learning goals

By the end of this lecture, you should be able to:

1. distinguish exact analytical answers from approximate numerical answers;
2. describe guess-and-check as candidate generation, evaluation, and acceptance;
3. implement exhaustive enumeration for a finite integer search;
4. explain how a loop makes progress and why it terminates;
5. define a tolerance with engineering meaning;
6. implement incremental search using a step size;
7. explain the trade-off among step size, accuracy, and running time;
8. add a maximum-iteration safeguard and report success or failure honestly;
9. identify cases where a different search algorithm is needed.

NAZARBAYEV UNIVERSITY

From repetition to search



Guess-and-check is an algorithmic pattern

The “guess” is not random intuition. The program follows a defined sequence of candidates, evaluates each one, and stops according to a stated rule.

NAZARBAYEV UNIVERSITY

Exact answers and approximate answers

Exact

An exact answer satisfies the mathematical statement without approximation.

$$\sqrt{9} = 3, \qquad 3^2 - 9 = 0.$$

Integer counts and some algebraic calculations can be exact in the model.

Approximate

An approximate answer is accepted because its error evidence satisfies a requirement.

$\sqrt{10} \approx 3.1623$, $|3.1623^2 - 10|$ is small.

Most measured and simulated engineering quantities are approximate.

More digits do not create more truth

Numerical precision should be consistent with the model, data, units, and decision being supported.

NAZARBAYEV UNIVERSITY

Opening problem: how should a computer find $\sqrt{10}$?

Declarative statement

The desired value x satisfies

$$x^2 = 10.$$

This describes what a square root is, but not how to calculate one.

Imperative question

What sequence of steps can generate candidates, check them, and decide when one is close enough?

Guess	Guess squared
3.0	9.00
3.1	9.61
3.2	10.24
3.16	9.9856

Propose the next guess

NAZARBAYEV UNIVERSITY

The four parts of a numerical search

Part	Question	Square-root example
Candidate	What value will be tested?	current guess g
Evaluation	What model result follows?	g^2 or residual $g^2 - 10$
Acceptance	What counts as good enough?	$\lvert g^2 - 10 \rvert < \varepsilon$
Update	How is the next candidate chosen?	$g \leftarrow g + \Delta g$
Stopping safety	What prevents endless work?	range bound or maximum iterations

A complete specification

A numerical algorithm is incomplete until its candidate set, acceptance rule, update, and termination behavior are clear.

NAZARBAYEV UNIVERSITY

Exhaustive enumeration: an exact integer cube root

```
x = 125
candidate = 0

while candidate**3 < x:
    candidate += 1

if candidate**3 == x:
    print(f"Cube root = {candidate}")
else:
    print("No integer cube root")
```

Why exhaustive?

The program tests every nonnegative integer candidate in order until the cube reaches or exceeds the target.

For $x = 125$

Candidates 0, 1, 2, 3, 4, and 5 are considered. The exact test succeeds at 5.

NAZARBAYEV UNIVERSITY

Hand trace the search state

Test number	candidate	candidate^3	Loop condition for $x = 27$
1	0	0	True
2	1	1	True
3	2	8	True
4	3	27	False

The state changes predictably

Each pass increases `candidate` by one. When the loop stops, the program checks whether it landed exactly on the target or passed it.

Trace $x = 20$ - 3 min

Progress and termination

Progress measure

Before reaching the target, one useful measure is

$$D = |x| - \texttt{candidate}^3.$$

As the candidate increases, D moves toward zero or becomes negative.

A loop must not merely repeat

Ask:

- Which variable changes?
- Does the change move toward stopping?
- Is the stopping state reachable?
- What happens for unusual inputs?

Engineering habit

A progress argument is evidence that the algorithm will finish; it is not an optional comment after the code happens to work.

NAZARBAYEV UNIVERSITY

Extend the exact search to negative cubes

```
x = -125
candidate = 0
magnitude = abs(x)

while candidate**3 < magnitude:
    candidate += 1

if candidate**3 == magnitude:
    if x < 0:
        candidate = -candidate
    print(f"Integer cube root = {candidate}")
else:
    print("No integer cube root")
```

Separate two ideas

1. search the nonnegative magnitude;
2. restore the sign if an exact odd root exists.

Model knowledge matters

Negative numbers have real cube roots, but not real square roots.

NAZARBAYEV UNIVERSITY

Debugging activity: why does this loop never finish?

```
x = 64
candidate = 0

while candidate**3 < x:
    candidate = candidate

print(candidate)
```

Diagnose before editing

1. Which value controls the loop test?
2. What is its value after each pass?
3. What one-line print statement would reveal the problem?
4. What is the smallest repair?

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Approximation requires an acceptance rule

Residual

For the target $g^2 = 10$, define

$$r(g) = g^2 - 10.$$

An exact root has $r(g) = 0$. A numerical result may be accepted when

$$|r(g)| < \varepsilon.$$

Tolerance ε

The tolerance is a chosen acceptable level for the residual. It answers: "How close to the target is sufficient for this task?"

State the quantity and unit

A temperature residual may be in °C; a force residual may be in N; a squared-root residual has the units of the squared quantity.

NAZARBAYEV UNIVERSITY

Residual error is not always the same as input error

Function-space criterion

$$|f(x)| < \varepsilon_f$$

controls how closely the model output meets the target.

Input-space criterion

$$|x - x^*| < \varepsilon_x$$

controls how close the input is to the true solution x^*, which is usually unknown.

Sensitivity matters

The same residual can correspond to different input errors when the model is steep or flat. Later methods can also control the width of an interval known to contain the answer.

NAZARBAYEV UNIVERSITY

Incremental search for an approximate square root

```
target = 10.0
guess = 0.0
step = 0.001
tolerance = 0.01
iterations = 0
maximum_iterations = 10_000

while (abs(guess**2 - target) > tolerance
       and iterations < maximum_iterations):
    guess += step
    iterations += 1

residual = abs(guess**2 - target)
```

What the loop does

It samples the candidate line at spacing $\Delta g = 0.001$ and stops after success or a finite work budget.

The result still needs a report

The code must distinguish an accepted approximation from termination at the iteration limit.

NAZARBAYEV UNIVERSITY

Report success or controlled failure

```
if residual <= tolerance:
    print(f"Approximation = {guess:.4f}")
    print(f"Residual = {residual:.6f}")
    print(f"Iterations = {iterations}")
else:
    print("No acceptable value found")
    print(f"Final residual = {residual:.6f}")
```

Evidence of success

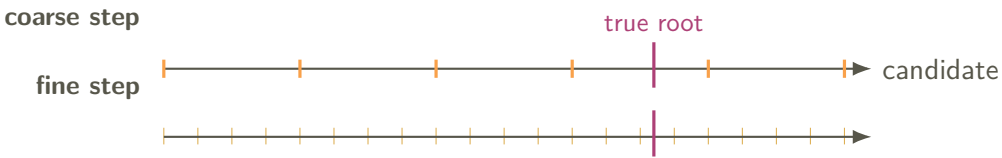
Report the candidate, residual, and computational effort.

Evidence of failure

Do not print the last candidate as though it were a solution when the acceptance condition was not met.

NAZARBAYEV UNIVERSITY

Step size controls resolution and cost



Large step

Fewer evaluations, but the sequence may jump over every acceptable candidate.

Small step

Finer resolution, but many more evaluations over the same interval.

NAZARBAYEV UNIVERSITY

A program can run correctly and still miss the answer

```
target = 10.0
guess = 0.0
step = 0.1
tolerance = 0.001

while guess <= 4.0:
    if abs(guess**2 - target) < tolerance:
        print("Found", guess)
        break
    guess += step
```

The sequence near the root

$$3.1^2 = 9.61, \qquad 3.2^2 = 10.24.$$

Neither residual is below 0.001. The acceptable region can lie between sampled points.

Classification

This is a numerical-design issue, not a syntax error.

Count the work as well as the answer

Approximate number of trials

Searching from a to b with step h requires roughly

$$N \approx \frac{b-a}{h}$$

candidate evaluations in the worst case.

Consequence

Reducing the step by a factor of 10 usually multiplies the work by about 10 over the same interval.

Record `iterations`

The number of guesses is evidence about method efficiency and helps diagnose unexpectedly long runs.

NAZARBAYEV UNIVERSITY

Use two stopping conditions

```
while (error > tolerance
       and iterations < maximum_iterations):
    # generate and evaluate next guess
    iterations += 1
```

Success condition

The residual or another error indicator becomes acceptable.

Safety condition

The program stops after a finite budget if the target is unreachable or the search design is poor.

Report honestly

Reaching the iteration limit is not a numerical solution. It is a controlled failure that should be communicated.

NAZARBAYEV UNIVERSITY

Why exact floating-point equality is fragile

```
x = 0.0

for _ in range(10):
    x += 0.1

print(x)
print(x == 1.0)
print(abs(x - 1.0) < 1e-12)
```

Typical result

```
0.9999999999999999
False
True
```

Better numerical question

Ask whether values are close enough for a justified tolerance, rather than whether their stored binary representations are identical.

NAZARBAYEV UNIVERSITY

Engineering example: minimum discrete area

```
force_n = 8000.0
allowable_stress_mpa = 150.0

area_mm2 = 10.0
area_step_mm2 = 5.0
stress_mpa = force_n / area_mm2  # N/mm^2 = MPa

while stress_mpa > allowable_stress_mpa:
    area_mm2 += area_step_mm2
    stress_mpa = force_n / area_mm2

print(f"Selected area = {area_mm2:.1f} mm^2")
print(f"Stress = {stress_mpa:.2f} MPa")
```

What is being approximated?

The exact threshold area is $8000/150 \approx 53.33\,\mathrm{mm}^2$, but the available design grid uses increments of $5\,\mathrm{mm}^2$.

Expected selection

The first tested safe area is $55\,\mathrm{mm}^2$.

NAZARBAYEV UNIVERSITY

When guess-and-check is a good first method

Good use cases

- small finite candidate sets;
- discrete standard sizes;
- simple prototypes and reference solutions;
- cases where transparency matters more than speed;
- checking a more advanced method.

Warning signs

- an extremely small step is required;
- the valid interval is large;
- the method repeatedly skips acceptable values;
- millions of evaluations are needed;
- the model provides directional information that is being ignored.

Next lecture

Bisection uses ordering and sign information to discard half of a valid interval after every evaluation.

NAZARBAYEV UNIVERSITY

Numerical-search checklist

1. **Target:** What condition are we trying to satisfy?
2. **Candidate set:** Which values are allowed or physically meaningful?
3. **Model:** What is calculated for each candidate?
4. **Acceptance:** Which residual or error measure is used?
5. **Tolerance:** What error is acceptable, and in which unit?
6. **Update:** How does the next candidate differ?
7. **Progress:** Why does the update move toward stopping?
8. **Safety:** What range or iteration limit prevents endless work?
9. **Evidence:** What value, residual, iteration count, and limitation will be reported?

NAZARBAYEV UNIVERSITY

Exit ticket: diagnose the search design

```
target = 2.0
guess = 0.0
step = 0.5

while guess**2 != target:
    guess += step

print(guess)
```

Submit four responses

1. Will the loop terminate?
2. Which candidate values occur near $\sqrt{2}$?
3. Replace exact equality with an acceptance rule.
4. Add one safety condition.

Individual - 4 min

Key message

Approximation becomes engineering computation only when the target, tolerance, update, termination, and evidence are explicit.

NAZARBAYEV UNIVERSITY

LECTURE 9

The Bisection Method for Engineering Root Finding

Using a valid interval, sign information, and repeated halving to obtain a controlled approximation

NAZARBAYEV UNIVERSITY

Lecture 9 learning goals

By the end of this lecture, you should be able to:

1. rewrite an engineering target condition in the form $f(x) = 0$;
2. explain what it means for a root to be bracketed;
3. state the continuity and sign-change assumptions used by bisection;
4. perform several bisection iterations by hand;
5. implement bisection using variables, conditionals, and a loop;
6. preserve the bracketing invariant when updating endpoints;
7. choose and interpret residual- and interval-based stopping criteria;
8. estimate the number of iterations required;
9. recognize invalid brackets, discontinuities, and roots without sign changes;
10. communicate a numerical result with error evidence and engineering limitations.

NAZARBAYEV UNIVERSITY

Retrieval warm-up: what is wrong with this search?

```
target = 10.0
guess = 0.0
step = 0.000001

while abs(guess**2 - target) > 1e-6:
    guess += step

print(guess)
```

Discuss

1. Is the method conceptually valid?
2. Approximately how many guesses are needed?
3. Which information about the target is ignored?
4. What safety condition is missing?

Pairs - 3 min

NAZARBAYEV UNIVERSITY

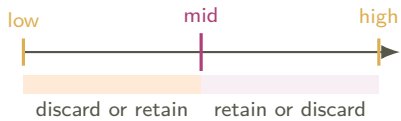
Opening idea: use order to discard possibilities

Searching a sorted dictionary

You do not inspect every page from the beginning. You open near the middle, decide whether the word must be earlier or later, and discard the impossible half.

Searching an ordered numerical interval

Evaluate the midpoint, decide which half can still contain the target, and repeat on the smaller interval.



Engineering targets become roots

Engineering question	Root-finding form
When does temperature reach T_{safe}?	$f(t) = T(t) - T_{\text{safe}} = 0$
Which area makes stress equal the allowable value?	$f(A) = \sigma(A) - \sigma_{\text{allow}} = 0$
Which resistance gives a target current?	$f(R) = I(R) - I_{\text{target}} = 0$
When does projectile height reach the ground?	$f(t) = h(t) - 0 = 0$
Which operating point balances generation and loss?	$f(x) = \text{generation}(x) - \text{loss}(x) = 0$

Root formulation rule

Define the residual as **calculated value minus target value**. The desired operating condition occurs when the residual is zero.

What is a bracketed root?

Bracket

An interval $[a, b]$ brackets a root when the available evidence guarantees that at least one root lies inside it.

Common sign-change test

For a continuous function,

$$f(a)f(b) < 0$$

means that the endpoint residuals have opposite signs, so the graph crosses zero somewhere between them.

Endpoint root

If $f(a) = 0$ or $f(b) = 0$, an endpoint is already an exact root. Therefore a practical validation often accepts

$$f(a)f(b) \leq 0.$$

Example bracket for $x^2 - 10 = 0$

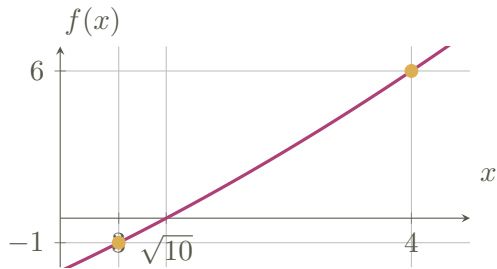
$$f(x) = x^2 - 10$$

$$f(3) = -1, \qquad f(4) = 6.$$

$$f(3)f(4) = -6 < 0.$$

Conclusion

A positive root lies in $[3, 4]$.



NAZARBAYEV UNIVERSITY

Activity: which intervals are valid sign-changing brackets?

For $f(x) = x^2 - 10$, calculate or reason about the endpoint signs.

Interval	$f(a)$ sign	$f(b)$ sign	Valid for bisection?
$[0, 2]$			
$[2, 4]$			
$[3, 4]$			
$[-4, -3]$			
$[-2, 2]$			

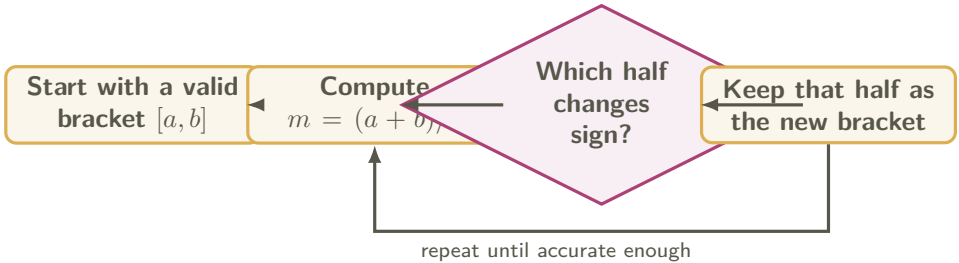
Do not guess from interval location alone

Evaluate the residuals. A valid bracket is evidence, not an impression that the interval "looks wide enough."

Small groups - 4 min

NAZARBAYEV UNIVERSITY

The bisection invariant



Invariant

At the start of every iteration, the current interval still brackets a root. The update must preserve this fact.

NAZARBAYEV UNIVERSITY

One bisection step by hand

For $f(x) = x^2 - 10$ with $[a, b] = [3, 4]$:

1. Midpoint:
$$m = \frac{3+4}{2} = 3.5.$$
2. Midpoint residual:
$$f(3.5) = 3.5^2 - 10 = 2.25 > 0.$$
3. Endpoint signs:
$$f(3) < 0, \qquad f(3.5) > 0.$$
4. New bracket:
$$[a, b] = [3, 3.5].$$

What was gained?

The bracket width falls from 1 to 0.5 after one midpoint evaluation.

NAZARBAYEV UNIVERSITY

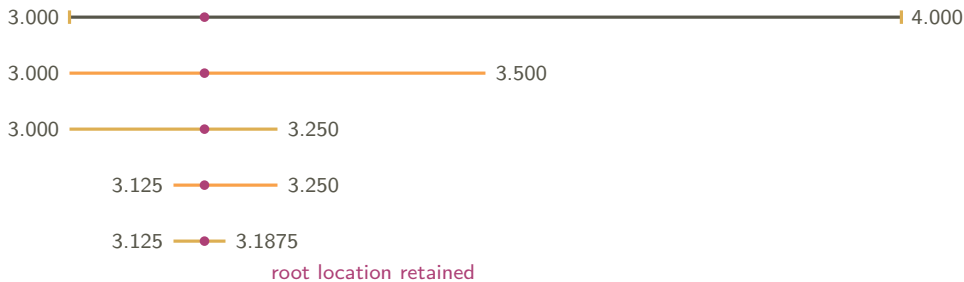
Trace the first five midpoint decisions

Iteration	low	high	midpoint	$f(\text{mid})$	retained interval
1	3.00000	4.00000	3.50000	$+2.25000$	$[3.00000, 3.50000]$
2	3.00000	3.50000	3.25000	$+0.56250$	$[3.00000, 3.25000]$
3	3.00000	3.25000	3.12500	-0.23438	$[3.12500, 3.25000]$
4	3.12500	3.25000	3.18750	$+0.16016$	$[3.12500, 3.18750]$
5	3.12500	3.18750	3.15625	-0.03809	$[3.15625, 3.18750]$

Read the table as evidence

The signs alternate as needed, the root remains bracketed, and the interval width is halved each iteration.

Visualizing repeated halving



The method does not need to know the exact root

It needs only a valid bracket and a reliable way to determine which subinterval preserves the sign change.

Bisection pseudocode

Algorithm

1. Choose low and high and verify a sign-changing bracket.
2. Accept an endpoint if its residual is already zero.
3. Compute the midpoint and its residual.
4. Retain the half whose endpoints preserve the sign change.
5. Repeat until the interval or residual meets the tolerance.
6. Report the estimate, interval, residual, and iterations.

Critical design requirement

Update from sign evidence, not merely because a midpoint appears “too large.”

NAZARBAYEV UNIVERSITY

Bisection setup and bracket check

```
target = 10.0
low = 3.0
high = 4.0
tolerance_x = 1e-6
maximum_iterations = 100
iterations = 0

f_low = low**2 - target
f_high = high**2 - target

if f_low * f_high > 0:
    print("Invalid bracket")
```

Before the loop

- define the target equation;
- choose the interval;
- evaluate both endpoint residuals;
- reject a non-sign-changing bracket.

Endpoint roots

A production version should also check whether $f(\text{low}) = 0$ or $f(\text{high}) = 0$ before iterating.

NAZARBAYEV UNIVERSITY

Bisection loop: preserve the bracket

```
while (high - low > tolerance_x
       and iterations < maximum_iterations):
    midpoint = (low + high) / 2
    f_mid = midpoint**2 - target

    if f_low * f_mid <= 0:
        high = midpoint
        f_high = f_mid
    else:
        low = midpoint
        f_low = f_mid

    iterations += 1
```

Invariant check

After either update, the endpoint residuals still have opposite signs or one endpoint is a root.

NAZARBAYEV UNIVERSITY

Finish the report: estimate and evidence

```
estimate = (low + high) / 2
residual = estimate**2 - target
interval_width = high - low

print(f"Estimate = {estimate:.8f}")
print(f"Residual = {residual:.3e}")
print(f"Interval width = {interval_width:.3e}")
print(f"Iterations = {iterations}")
```

Why report all four?

- estimate: proposed answer;
- residual: target mismatch;
- interval width: location uncertainty evidence;
- iterations: computational effort.

Two useful stopping criteria

Residual criterion

$$|f(m)| \le \varepsilon_f.$$

Stops when the model output is close to the target.

Interval-width criterion

$$b - a \le \varepsilon_x.$$

Stops when the bracket containing the root is sufficiently narrow.

Limitation

A small residual does not always imply a small input error when the function is very flat.

Midpoint guarantee

If the root remains bracketed, the midpoint is at most $(b-a)/2$ from that root.

NAZARBAYEV UNIVERSITY

How many iterations will bisection need?

After n iterations, the width is $L_n = L_0/2^n$, where $L_0 = b_0 - a_0$.

Required iterations for a width tolerance

To ensure $L_n \le \varepsilon_x$,

$$n \ge \left\lceil \log_2\left(\frac{L_0}{\varepsilon_x}\right) \right\rceil.$$

Example

Starting with width 1 and requiring width at most 10^{-6} needs

$$\lceil \log_2(10^6) \rceil = 20.$$

NAZARBAYEV UNIVERSITY

Incremental search versus bisection

	Incremental search on $[3, 4]$	**Bisection on** $[3, 4]$
Update	add a fixed step	halve the retained interval
Information used	candidate order only	order plus endpoint signs
Resolution 10^{-6}	about 10^6 samples	20 interval halvings
Risk	may skip acceptance region	converges if assumptions hold
Error evidence	depends on step and residual	explicit bracket width
Transparency	very simple	still simple, but more structured

Algorithm choice matters more than machine speed

A better use of mathematical structure can reduce millions of evaluations to a few tens.

NAZARBAYEV UNIVERSITY

Boundary case: an endpoint is already the root

```
target = 9.0
low = 3.0
high = 5.0

f_low = low**2 - target
f_high = high**2 - target

if f_low == 0:
    estimate = low
elif f_high == 0:
    estimate = high
elif f_low * f_high < 0:
    print("Proceed with bisection")
else:
    print("No sign-changing bracket")
```

For this case

$f(3) = 0$, so no iteration is needed.

Why check first?

A product test using only < 0 would reject an interval whose endpoint is exactly a valid root.

A sign change requires continuity

Counterexample

$$f(x) = \frac{1}{x}, \qquad [-1, 1].$$

The endpoint values have opposite signs, but f is undefined at $x = 0$ and has no root.

Engineering interpretation

Discontinuities may arise from division by zero, switching models, phase changes, contact events, or invalid domains. A sign change across a discontinuity is not evidence of a physical balance point.

Assumption audit

Before using bisection, ask whether the residual is defined and continuous throughout the proposed interval.

NAZARBAYEV UNIVERSITY

A root may exist without a sign change

Touching root

$$f(x) = (x-2)^2.$$

On $[0, 4]$, both endpoint values are positive even though $x = 2$ is a root. The graph touches zero and turns around.

Even number of crossings

$$g(x) = (x-1)(x-3).$$

On $[0, 4]$, the endpoint signs match although two roots lie inside.

Consequence

Failure of the endpoint sign-change test does not prove that no root exists. It means this interval does not provide the particular evidence required by basic bisection.

NAZARBAYEV UNIVERSITY

Engineering example: when is a component safe to handle?

Cooling model

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-kt},$$

with

$$T_\infty = 25^\circ\mathrm{C}, \quad T_0 = 100^\circ\mathrm{C}, \quad k = 0.04\,\mathrm{s}^{-1}.$$

Target condition

Find the time when $T(t) = 40^\circ\mathrm{C}$. Define

$$f(t) = T(t) - 40.$$

The required time is a root of f.

Bracket the cooling time before iterating

At $t = 0$:

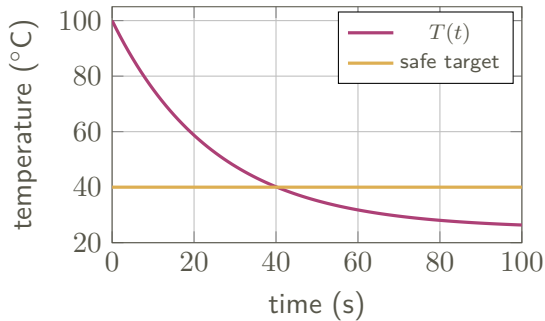
$$T(0) = 100^\circ\mathrm{C},$$

$$f(0) = 60^\circ\mathrm{C} > 0.$$

At $t = 100\,\mathrm{s}$:

$$T(100) \approx 26.37^\circ\mathrm{C},$$

$$f(100) \approx -13.63^\circ\mathrm{C} < 0.$$



Bracket conclusion

The continuous cooling model changes from above to below the target on $[0, 100]$ s.

NAZARBAYEV UNIVERSITY

Cooling example: model and numerical settings

```
import math

ambient_c = 25.0
initial_c = 100.0
cooling_rate_per_s = 0.04
safe_c = 40.0

low_s = 0.0
high_s = 100.0
tolerance_s = 0.01
maximum_iterations = 100
iterations = 0
```

Meaning

- the interval is in seconds;
- the residual is in °C;
- the interval-width tolerance controls time uncertainty;
- the iteration limit prevents endless work.

NAZARBAYEV UNIVERSITY

Cooling example: bisection loop

```
low_residual = (ambient_c + (initial_c - ambient_c)
                * math.exp(-cooling_rate_per_s * low_s) - safe_c)

while (high_s - low_s > tolerance_s
       and iterations < maximum_iterations):
    mid_s = (low_s + high_s) / 2
    mid_residual = (ambient_c + (initial_c - ambient_c)
                    * math.exp(-cooling_rate_per_s * mid_s) - safe_c)

    if low_residual * mid_residual <= 0:
        high_s = mid_s
    else:
        low_s = mid_s
        low_residual = mid_residual

    iterations += 1
```

NAZARBAYEV UNIVERSITY

Report and interpret the cooling result

```
estimate_s = (low_s + high_s) / 2
estimate_c = (ambient_c + (initial_c - ambient_c
    )
              * math.exp(-cooling_rate_per_s *
    estimate_s))

print(f"Safe-time estimate = {estimate_s:.2f} s"
    )
print(f"Temperature = {estimate_c:.3f} deg C")
print(f"Final bracket width = {high_s-low_s:.4f}
     s")
print(f"Iterations = {iterations}")
```

Expected result

The crossing occurs near 40.24 s. The final time uncertainty from the bracket is no more than half its width.

Not a complete safety guarantee

The conclusion depends on the cooling model, parameter k, sensor accuracy, ambient temperature, and the definition of "safe."

NAZARBAYEV UNIVERSITY

Debugging clinic: repair the bisection logic

```
low = 3.0
high = 4.0
mid = (low + high) / 2

while high - low > 1e-6:
    f_mid = mid**2 - 10

    if f_mid < 0:
        high = mid
    else:
        low = mid

print(mid)
```

Find at least three defects

1. Is the endpoint update direction correct?
2. Is the midpoint recomputed?
3. Is the bracket sign evidence preserved?
4. Will the loop make progress?

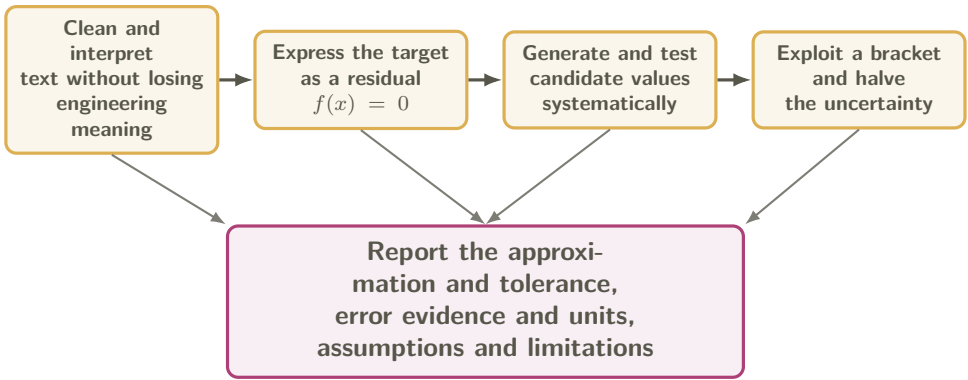
Pairs - 5 min

NAZARBAYEV UNIVERSITY

Bisection method checklist

1. **Residual:** Is the engineering target correctly written as $f(x) = 0$?
2. **Domain:** Is f defined throughout the interval?
3. **Continuity:** Is a sign change meaningful for this model?
4. **Bracket:** Have the endpoint residuals been evaluated and checked?
5. **Invariant:** Does every endpoint update preserve a sign-changing interval?
6. **Tolerance:** Does the stopping criterion control the quantity that matters?
7. **Safety:** Is there a maximum iteration count?
8. **Evidence:** Are the estimate, residual, bracket width, and iterations reported?
9. **Interpretation:** Are units, assumptions, physical reasonableness, and limitations stated?

Week 3 concept map



NAZARBAYEV UNIVERSITY

Exit ticket: choose and justify a numerical strategy

A projectile-height model is

$$h(t) = 30 + 18t - 4.905t^2.$$

Find the impact time when $h(t) = 0$.

Your task

1. define the residual $f(t)$;
2. propose a physical interval for t;
3. state the endpoint evidence required;
4. choose a time tolerance;
5. describe one bisection update.

Final reflection

What does the numerical method determine, and what must the engineer still decide about the model, initial conditions, and physical situation?

Individual - 5 min

Week 3 quick reference

Strings

`s[i]`	character at index i
`s[a:b]`	slice from a through b-1
`s.strip()`	remove surrounding whitespace
`s.lower()`	return lowercase text
`s.split(sep)`	divide text at a separator
`sep.join(parts)`	combine strings with a separator

Numerical search

`abs(f(x)) < tol`	residual-based acceptance
`x += step`	incremental-search update
`f(a) * f(b) < 0`	sign-changing bracket
`mid = (a+b)/2`	bisection midpoint
`max_iterations`	safety limit on repetition
`b-a < tol_x`	interval-width stopping test

The engineering questions

What information does the text encode? What error is acceptable? Why must the search terminate? What evidence makes the approximation trustworthy?

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 3.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapters 4 and 12 and Appendix A.
- J. V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Sections 2.3 and 3.1–3.4.

Primary preparation for students

Run each short example, predict the output before execution, trace search variables by hand, and explain the tolerance and stopping rule in engineering language.

NAZARBAYEV UNIVERSITY

Approximation is not guessing without discipline

Define the target. Preserve the bracket. Control the error. Check the physics. Report only the precision the evidence supports.

ENG 101 - Programming for Engineers